

07 — Frontend Architecture

See also: [DESIGN_SYSTEM_DOCUMENTATION.md](#) for tokens, type scale, color palette, spacing.

Source: [frontend-react/](#) (Vite + React 19 + TypeScript).

1. Tooling

Tool	Use
Vite 6	Dev server, build, HMR
TypeScript 5	Type safety
React 19	UI library
react-router-dom 7	Client routing
Zustand 5	Lightweight state
Vitest 3 + RTL + jsdom	Unit/component tests
Playwright	E2E from repo root
ESLint + Prettier	Lint / format

2. Folder layout ([frontend-react/src](#))

api/	REST clients per domain (auth, products, cart, orders, ...)
components/	Reusable UI (Header, Footer, ProductCard, VariantSelector, ...)
config/	env.ts (VITE_API_URL, etc.)
contexts/	React contexts (theme, toast)
hooks/	useDebounce, useAuth, useCart, useFavorites, ...
lib/	fetch client + auth interceptor + currency/VAT formatters
pages/	Route components (one folder per page or grouped area)
stores/	Zustand stores (auth, cart, favorites, home)
styles/	CSS modules + design tokens (tokens.css)
test/	Vitest setup + helpers
types/	Shared TS types

3. Routing

`react-router-dom` v7. Top-level pages (see [frontend-react/src/pages/](#)):

Path	Component
/	HomePage
/landing/:slug	LandingPage
/products	ProductListPage
/products/:slug	ProductDetailPage
/categories	CategoriesPage
/categories/:slug	category landing
/brands	BrandsPage
/brands/:slug	brand listing
/cart	CartPage
/checkout	CheckoutPage
/payment-callback	PaymentCallbackPage
/favorites	FavoritesPage
/contact , /faq , /privacy-policy , /terms , /bulk-orders	static pages
/account/*	profile, addresses, orders, returns, loyalty
/auth/*	login, register, forgot/reset password, verify-email
/admin/*	admin shell — RBAC-gated wrapper
*	NotFoundPage

Guarded routes redirect through `useAuth()` to `/auth/login?next=...`.

4. State management

Zustand stores live in `src/stores/`. They are intentionally small and domain-focused:

Store	Responsibility
<code>authStore</code>	<code>user</code> , <code>tokens</code> , <code>login()</code> , <code>logout()</code> , <code>refresh()</code> , persisted to <code>localStorage</code>
<code>cartStore</code>	active cart, totals, optimistic add/update/remove with server reconciliation
<code>favoritesStore</code>	favorite product IDs and detail cache
<code>homeStore</code>	cached home payload (banners, sections, navigation)

Stores never call `fetch` directly; they delegate to API modules in `src/api/`.

5. API layer

A central `lib/apiClient.ts` (or equivalent) wraps `fetch`:

- Reads `import.meta.env.VITE_API_URL` (e.g., `https://backend-...azurecontainerapps.io/api`).
- Adds `Authorization: Bearer <accessToken>` when present.
- On `401`, calls `/auth/refresh` once; retries the original request; if refresh fails, clears auth state and redirects to login.
- Surfaces server error envelope (`{ statusCode, message, error }`) to the caller as a typed `ApiError`.

Each domain module (`api/products.ts` , `api/orders.ts` , ...) exports typed functions that consume the client.

6. Component conventions

- **Pages** are thin: fetch via store/hook, render layout + sections.
- **Smart vs presentational** — heavy logic in hooks (`useProductDetail` , `useCheckout`); UI components are mostly pure props.
- **Accessibility** — semantic HTML, `aria-` where needed, focus management on dialogs.
- **Variant selector** — receives the current product + sibling array; hides axes not present (e.g., size button shown only when current product has a size). Tested in `VariantSelector.test.tsx`.

7. Styling

- CSS modules per component.
- Global tokens in `styles/tokens.css` (colors, spacing, type scale, radii, shadows). See [DESIGN_SYSTEM_DOCUMENTATION.md](#).
- Mobile-first; breakpoints `sm/md/lg/xl`.
- Currency rendering centralised in `lib/format.ts` — always shows AED to two decimals; VAT-inclusive on storefront, with optional VAT split shown in cart/checkout summaries.

8. Build & dev

```
cd frontend-react
npm install
npm run dev          # vite on http://localhost:5173
npm run build        # outputs dist/
npm run preview      # serve dist/ locally
npm run test         # vitest run
npm run test:watch
```

Required env vars (see [.env.example](#) if present):

- `VITE_API_URL` — base URL **including** the `/api` prefix.
- `VITE_STRIPE_PUBLIC_KEY` — Stripe publishable key.

9. Performance considerations

- Lazy-load admin routes via `React.lazy()` to keep the storefront bundle small.
- Image rendering uses Blob-served variants (`thumb` , `medium` , `large`) picked via `srcset` .
- Home payload comes pre-assembled from `/api/content/home` so the storefront makes few requests on first paint.

10. SEO

- React Router renders the SPA shell; SWA serves `index.html` with meta-tag fallbacks.
- Per-page titles & descriptions set with a `useDocumentMeta` hook.
- Sitemap and robots are static under `public/` .

11. Linking back to data

The complete database-consumption mapping (which page reads which table / endpoint) is captured in [FRONTEND_DB_CONSUMPTION_REPORT.md](#).